

PRÁCTICA 5-A

Ingeniería del Software II

(Pedro Cuesta y Hugo Silva)

Análisis y Diseño de Casos de Prueba

1. Diseño de Casos de Prueba (Caja Negra)

1.1. Función precio total (seguro)

A continuación se detallan las clases de equivalencia para los parámetros de la función del precio del seguro:

Parámetro	Clases válidas	Clases no válidas
fechaInicio	- fechaActual - 1 año < fechaInicio ≤ fechaActual - fechaInicio ≤ fechaActual - 1 año - fechaInicio > fechaActual	- null
cobertura	- TODO_RIESGO - TERCEROS_LUNAS - TERCEROS	- null
potencia	- (0, 89] - (89, 110] - (110, ∞)	- (-∞, 0]

Cuadro 1: Clases de equivalencia para Seguro

1.2. Función total seguros (cliente)

Clases de equivalencia para los parámetros de la función del total de seguros del cliente:

Parámetro	Clases válidas	Clases no válidas
minusvalía	true, false	-
lista de seguros	- ["Lista Vacía"] - ["Lista con objetos"]	- null

Cuadro 2: Clases de equivalencia para Cliente

1.3. Función añadir seguro (cliente)

Parámetro	Clases válidas	Clases no válidas
Seguros	- ["Lista Vacía"] - ["Lista con objetos"]	- null
Seguro	- Seguro	- null

Cuadro 3: Clases de equivalencia para Cliente

1.4. Función eliminar seguro (cliente)

Parámetro	Clases válidas	Clases no válidas
Seguros	- ["Lista con objetos"]	- ["Lista Vacía"] - null
Seguro	- Seguro	- Seguro que no está en la lista - null

Cuadro 4: Clases de equivalencia para Cliente

2. Análisis de Valores Límite (AVL)

2.1. Función Precio Total

Para la función del precio total, se han diseñado los siguientes valores límite:

- **Potencia:**
 - **(0, 89]:** 1 (límite inferior), 89 (límite superior), 45 (medio).
 - **[90, 110]:** 90 (límite inferior), 110 (límite superior), 100 (medio).
 - **(110, ∞):** 111 (límite inferior), 200 (medio).
 - **No válidos:** null.
- **Cobertura:**
 - TODO_RIESGO, TERCEROS_LUNAS, TERCEROS (válidos).
 - null (no válido).
- **FechaInicio:**
 - **fechaActual - 1 año < fechaInicio ≤ fechaActual**
 - **fechaInicio ≤ fechaActual - 1 año**
 - **fechaInicio > fechaActual**
 - **No válidos** null.

2.2. Función Precio Total Seguros (Cliente)

- **Minusvalía:** true / false (válido).
- **Lista de seguros:** 0 seguros (vacío), 1 seguro, más de 1 seguro (varios).

- **Función Añadir Seguros (Cliente)**

- []
- Seguros: [seguro1] Seguros:[seguro1, seguro2]
- Seguro1 (no está en el array)
- Null
- Null

2.3. Función Eliminar Seguros (Cliente)

- Seguros: [seguro1] Seguros:[seguro1, seguro2]
- Seguro1 (está en el array)
- Null
- []
- Null
- Seguro2 (no está en el array)

3. Definición de Casos de Prueba

3.1. Casos de Prueba Válidos - precio()

1. fechaActual - 1 año + 1 día, TODO RIESGO, 1
2. fechaActual - 6 meses, TERCEROS_LUNAS, 90
3. fechaActual, TERCEROS, 111
4. fechaActual - 3 años, TODO RIESGO, 50
5. fechaActual - 1 año, TERCEROS_LUNAS, 100
6. fechaActual + 1 día, TERCEROS, 125
7. fechaActual + 1 año, TODO RIESGO, 89
8. fechaActual, TERCEROS_LUNAS, 110

3.2. Casos de Prueba No Válidos - precio()

1. null, TERCEROS, 1
2. fechaActual, null, 90
3. fechaActual - 3 años, TODO RIESGO, 0
4. fechaActual + 1 día, TERCEROS_LUNAS, -15

3.3. Casos de Prueba Válidos - totalSeguros()

1. true, lista vacía (0 seguros)
2. false, lista con 1 seguro
3. true, lista con más de 1 seguro (2 seguros)

3.4. Casos de Prueba No Válidos - totalSeguros()

1. false, null (lista de seguros nula)

3.5. Casos de Prueba Válidos – añadirSeguro()

1. [], Seguro1
2. [Seguro1], Seguro2
3. [Seguro1,Seguro2], Seguro3

3.6. Casos de Prueba No Válidos – añadirSeguro()

1. [], null
2. null, Seguro1
3. [Seguro1], Seguro1

3.7. Casos de Prueba Válidos – eliminarSeguro()

1. [Seguro1], Seguro1
2. [Seguro1, Seguro2], Seguro1

3.8. Casos de Prueba Válidos – eliminarSeguro()

1. [Seguro1], null
2. [], Seguro1
3. Null, Seguro1
4. [Seguro2], Seguro1

4. Código en JUnit y Cobertura

A continuación programamos los tests correspondientes con las clases Seguro y Cliente, creando dos nuevas clases SegurosTests y ClientesTests. Ambas contendrán un test para el constructor, uno para los casos válidos y otro para los casos no válidos. Una vez creadas, ejecutaremos el comando mvn test dentro de la carpeta seguroscommon para ejecutar dichos tests. Con todo esto obtenemos los siguientes resultados:

```
PROBLEMS 9 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.006 s -- in es.unican.is2.SegurosTest
[INFO] Results:
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.267 s
[INFO] Finished at: 2026-04-28T20:55:31+02:00
[INFO] -----
PS C:\Users\pedro\IS2_2526\practica5\seguroscommon>
```

Figura 1: Resultado de los tests unitarios

En las capturas observamos que todos los tests se ejecutan de manera correcta, con cero *Failures* y cero *Errors*, por lo que no se observa ningún problema con las clases válidas y no válidas, y los valores propuestos para ellas.

Por último, comprobamos la cobertura de los tests creados utilizando la herramienta - un Tests with Coverage”de Visual Studio Code. Tras ejecutarlo, obtenemos los siguientes resultados:

File	Coverage
Seguro.java	77.94%

```
public class Seguro {
    public double precio() {
        // Si aún no ha empezado el seguro, precio = 0
        if (fechaInicio != null && LocalDate.now().isBefore(fechaInicio)) return 0;

        // Precio según la cobertura del seguro
        double precio = 0;
        if (cobertura == Cobertura.TODO_RIESGO) {
            precio = 1000;
        } else if (cobertura == Cobertura.TERCEROS_LUNAS) {
            precio = 600;
        } else if (cobertura == Cobertura.TERCEROS) {
            precio = 400;
        }

        // Incremento por potencia (multiplicamos directamente por 1.05 o 1.20)
        if (potencia >= 90 && potencia <= 110) {
            precio *= 1.05;
        } else if (potencia > 110) {
            precio *= 1.20;
        }

        // Descuento del 20% en el primer año (multiplicamos por 0.80)
        if (fechaInicio != null && LocalDate.now().isBefore(fechaInicio.plusYears(yearsToAdd: 1))) {
            precio *= 0.80;
        }
    }
}
```

Figura 2: Cobertura de la clase Seguro

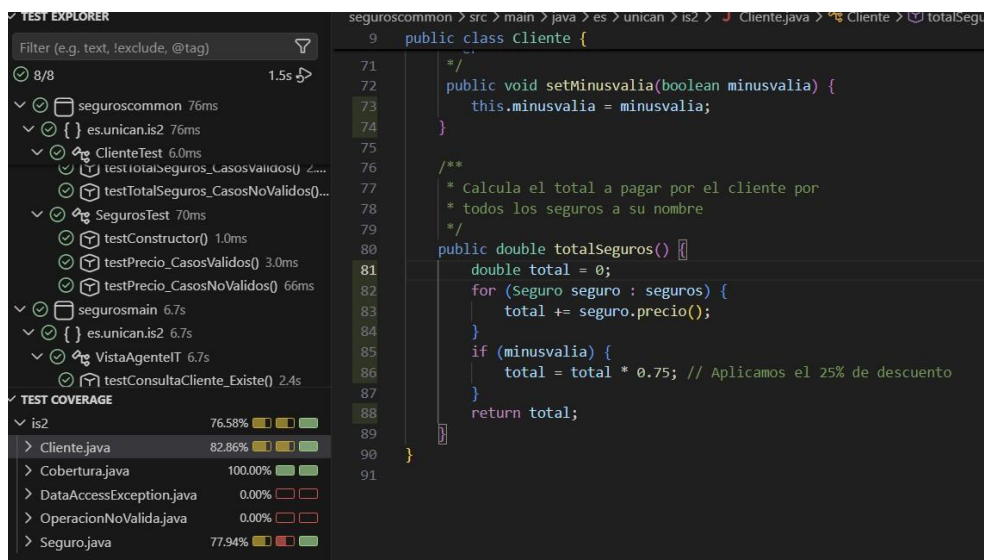


Figura 3: Cobertura de la clase Cliente

En ambas capturas se observa que las funciones `precio()` y `totalSeguros()` tienen todas las líneas de código de su implementación en color verde, por lo que no sugieren ningún error. Además, la cobertura total ronda el 60 % ya que no se han probado los métodos `get/set`.

5. Pruebas de Integración

En primer lugar, realizamos el diseño de los casos de prueba:

- **Cliente válido:** Entrada DNI = "111111111A".
Resultado esperado: Nombre = "Juan", Total a pagar = 1820.0 (Suma de sus 3 seguros según la base de datos).
- **Cliente no existente:** Entrada DNI = "00000000000Z".
Resultado esperado: El campo nombre muestra el mensaje "DNI No Válido".
- **Error en BBDD:** Entrada DNI = "vacío".
Resultado esperado: El campo nombre muestra el mensaje ".ERROR EN BBDD"

Posteriormente, seguimos con la implementación en JUnit utilizando `AssertJ`, creando la clase `VistaAgenteIT` (Integration Tests) y su correspondiente robot, configurando tanto las dependencias como el propio `pom.xml` de la carpeta `segurosmain`.

Después realizamos la ejecución y corregimos los siguientes errores:

1. **Error 1:** El botón de buscar leía un campo incorrecto `txtNombreCliente`.^{en} lugar de `txtDniCliente`. Se corrigió modificando dicho campo en el `ActionListener`.
2. **Error 2:** Cuando el cliente no existe en la BBDD (`c == null`), la interfaz mostraba el mensaje ".Error en BBDD". Se corrigió para mostrar "DNI No Válido".
3. **Error 3:** La base de datos no se iniciaba. Se solucionó añadiendo la dependencia de la base de datos H2 al `pom` de `segurosmain`.

Por último, utilizando el comando `mvn verify`, el robot ejecuta las pruebas resultando en **BUILD SUCCESS**.

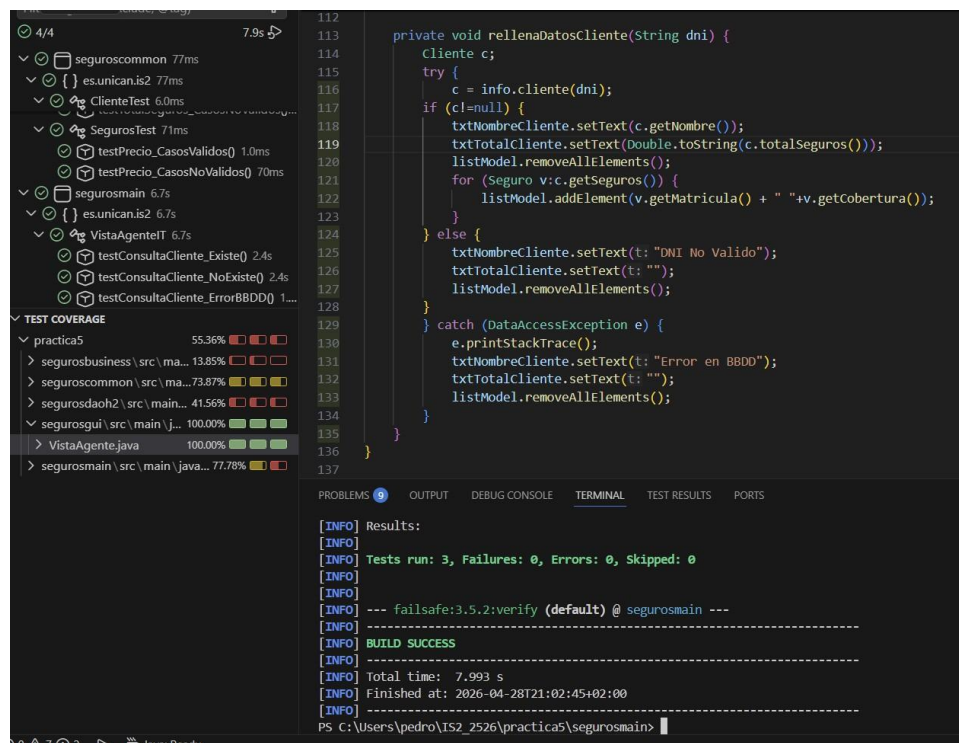


Figura 4: Resultados de Pruebas de Integración

Como se observa en la captura, los tests tienen un 100% de coverage sobre la clase `VistaAgente`, por lo que no hace falta implementar ningún test más.